

Distributed Systems

DAT520 - Spring 2026

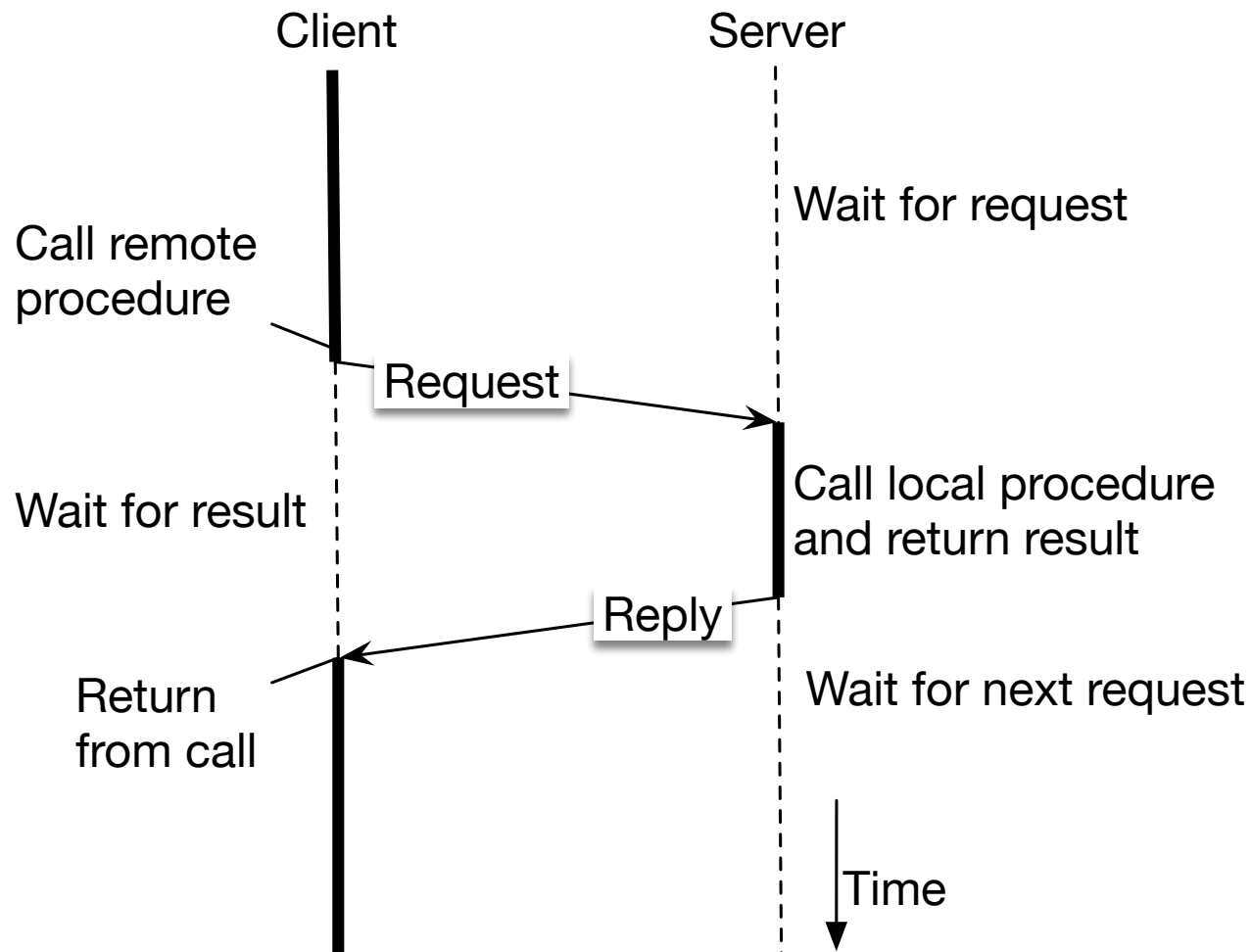
Remote Procedure Calls

Prof. Hein Meling

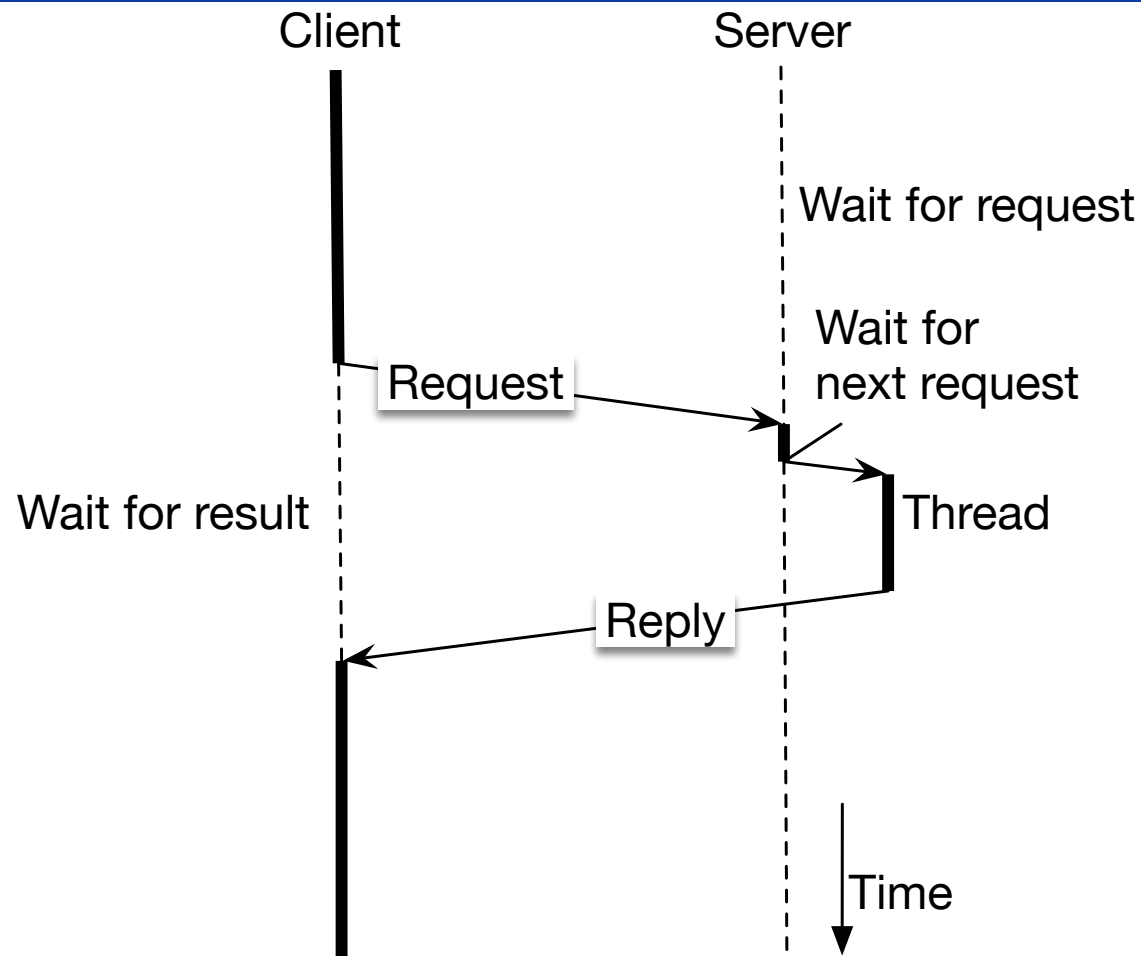


Remote Procedure Call

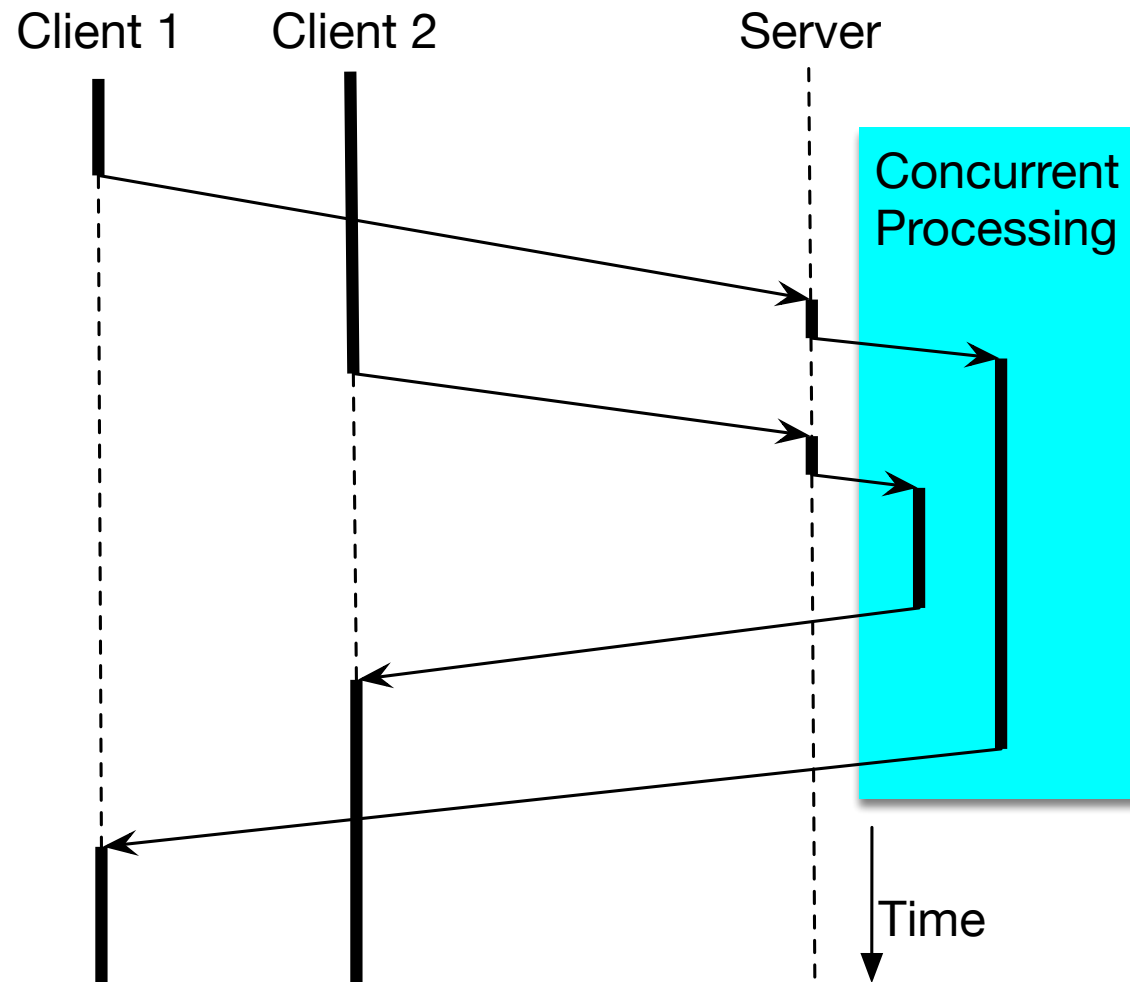
Remote Procedure Call



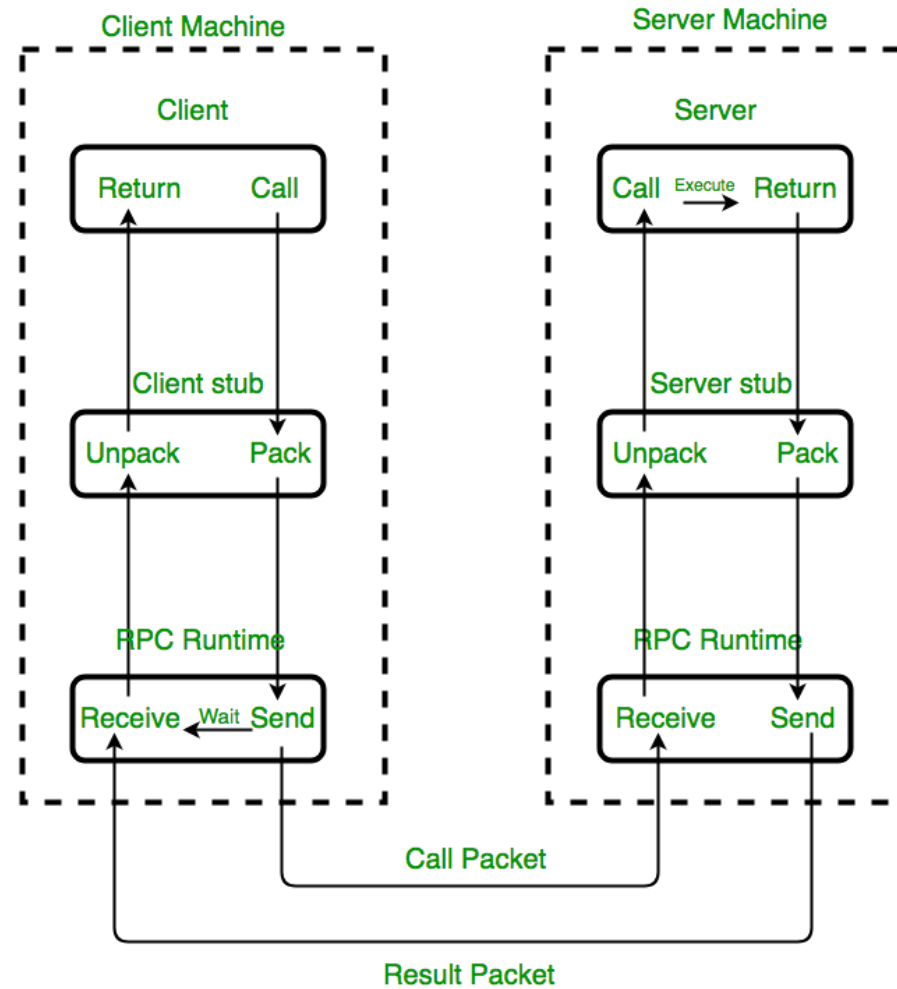
Remote Procedure Call



Remote Procedure Call

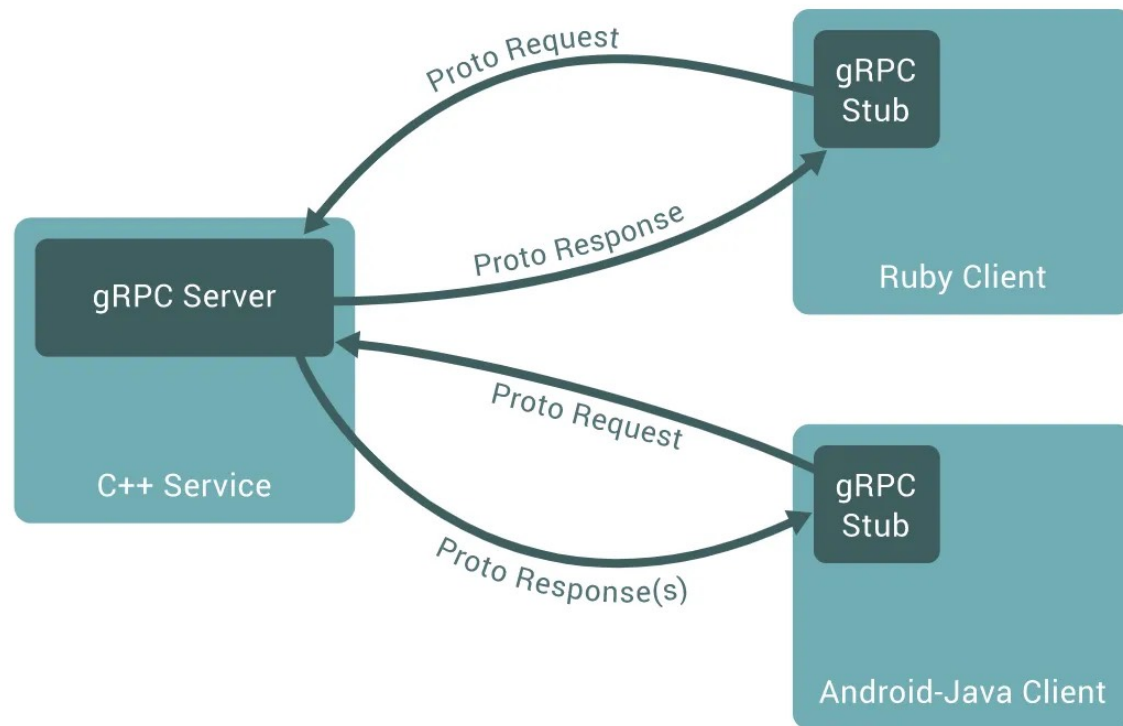


Remote Procedure Call



Google's gRPC Remote Procedure Call

gRPC is a recursive acronym



High-performance RPC framework

- Uses Protocol Buffers (protobuf) as its Interface Definition Language (IDL)
- Supports bi-directional streaming
- Uses HTTP/2 for transport, enabling multiplexed streams
- Strongly typed contracts between services with auto-generated code
- Widely used in microservices and distributed systems

Interface Definition Language (IDL)

- Specifies RPC methods and message content

```
service HelloService {  
    rpc SayHello (HelloRequest) returns (HelloResponse);  
}  
  
message HelloRequest {  
    string greeting = 1;  
}  
  
message HelloResponse {  
    string reply = 1;  
}
```

Four kinds of service methods

- Unary RPCs
 - Client sends a single request to the server
 - Get single response back
 - (like a normal function call)

```
rpc SayHello(HelloRequest) returns (HelloResponse);
```

Four kinds of service methods

- Server streaming RPCs
 - Client sends a request to the server
 - Gets a stream to read a sequence of messages back from the server
 - Client reads from the stream until there are no more messages
 - (guaranteed message ordering within an individual RPC call)

```
rpc LotsOfReplies(HelloRequest) returns (stream HelloResponse);
```

Four kinds of service methods

- Client streaming RPCs
 - Client sends a sequence of messages to the server using a stream
 - Client waits for the server to read the messages and return a response
 - (guaranteed message ordering within an individual RPC call)

```
rpc LotsOfGreetings(stream HelloRequest) returns (HelloResponse);
```

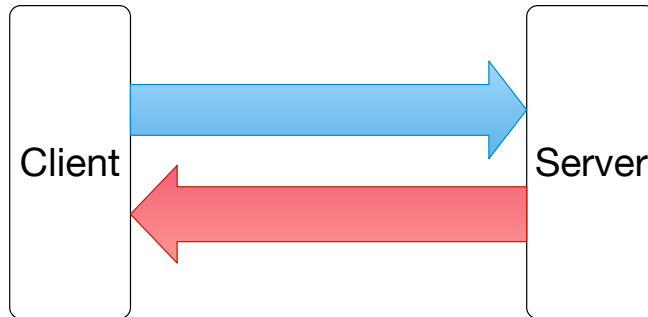
Four kinds of service methods

- Bidirectional streaming RPCs
 - Both sides send a sequence of messages using a read-write stream
 - The two streams operate independently
 - Clients and servers can read and write in whatever order they like
 - (order of messages in each stream is preserved)

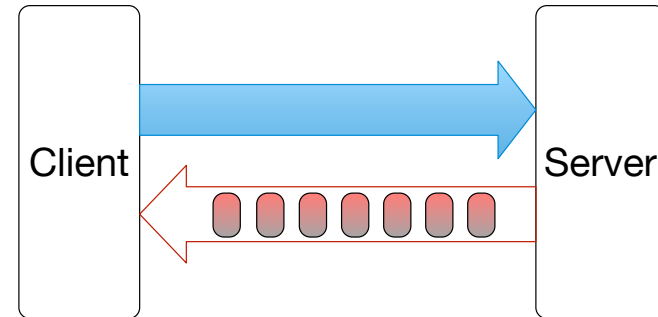
```
rpc BidiHello(stream HelloRequest) returns (stream HelloResponse);
```

gRPC Remote Procedure Calls

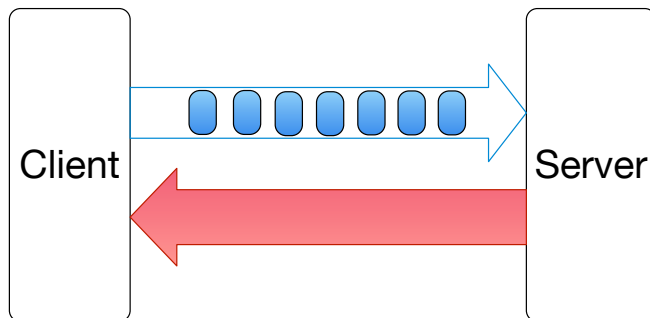
Unary



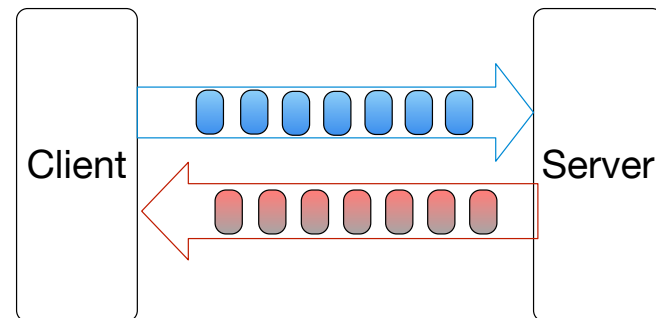
Server Streaming



Client Streaming



Bidirectional Streaming



Installing the toolchain

```
$ brew install protobuf
```

```
# Can generate code for: Java, C++, C#, Python, ...
```

```
$ protoc --version
```

```
# Go requires a separate plugin:
```

```
$ go install google.golang.org/protobuf/cmd/protoc-gen-go@latest
```

```
$ go install google.golang.org/grpc/cmd/protoc-gen-go-grpc@latest
```


Joke time

“gRPC errors are very polite:
they tell you exactly nothing...”

“REST is for humans.
gRPC is for machines—and
the occasional brave debugger.”

Why is gRPC so fast?

Because it skips the small
talk and gets straight to the
point with binary protobufs!

Questions?

- <https://grpc.io/>
- <https://buf.build/docs/introduction>
- <https://buf.build/>